

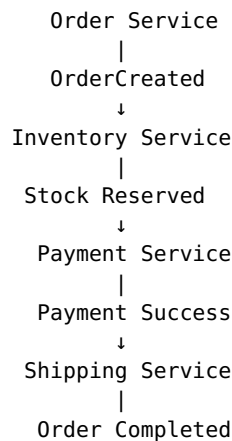
Saga - Choreography and Orchestration - Complete Interview Notes

5. SAGA — CHOREOGRAPHY

Saga is used when **one business transaction spans multiple microservices** and each service has its own database.

A normal database transaction cannot safely cover multiple independent microservice databases.

Example: Order → Inventory → Payment → Shipping



There is **no central coordinator** in choreography.

Each service:

1. Performs its own local transaction.
2. Publishes an event.
3. Another service listens to that event.
4. Performs its own transaction.
5. Publishes the next event.

1. Why Saga?

Suppose:

Order Service	→	SUCCESS
Inventory Service	→	SUCCESS
Payment Service	→	FAILURE

We can't simply do:

`@Transactional`

across all three services because:

Order DB ≠ Inventory DB ≠ Payment DB

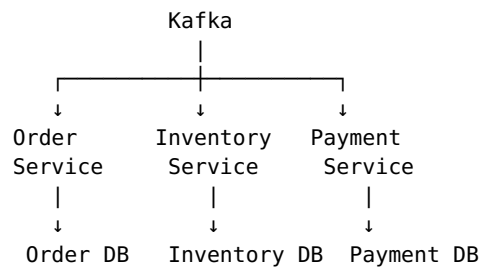
Instead, we use compensation.

Order Created

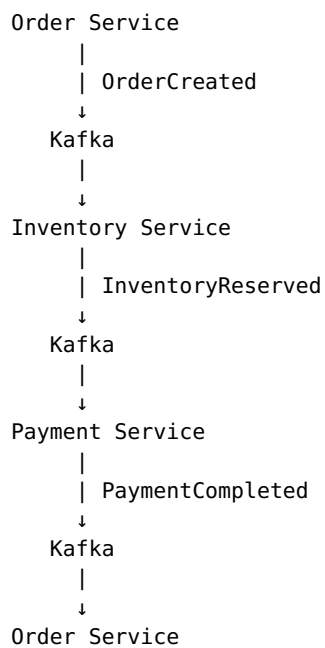
↓
Stock Reserved
↓
Payment FAILED
↓
Release Stock
↓
Cancel Order

This is called a **compensating action**.

2. Choreography architecture



Events drive the workflow.



3. Step 1 — Order Service

```
@Service
public class OrderService {

    private final OrderRepository orderRepository;
    private final KafkaTemplate<String, Object> kafkaTemplate;

    public OrderService(
        OrderRepository orderRepository,
        KafkaTemplate<String, Object> kafkaTemplate) {
        this.orderRepository = orderRepository;
        this.kafkaTemplate = kafkaTemplate;
    }

    @Transactional
    public void createOrder(Order order) {
```

```

        orderRepository.save(order);

        kafkaTemplate.send(
            "order-created",
            new OrderCreatedEvent(
                order.getId(),
                order.getItems()
            )
        );
    }
}

```

Conceptually:

```

Order DB
  ↓
Order CREATED
  ↓
order-created
  ↓
Kafka

```

Important

For production, don't rely on:

```

orderRepository.save(order);
kafkaTemplate.send(...);

```

as two independent writes.

Use **Transactional Outbox**:

```

@Transactional
├── Save Order
├── Save Outbox Event
│   ↓
│   COMMIT
│   ↓
│   Outbox Publisher
│   ↓
│   Kafka

```

This connects directly with the Outbox pattern you studied earlier.

4. Step 2 — Inventory Service

Inventory listens for:

order-created

```

@Service
public class InventorySagaListener {

    private final InventoryService inventoryService;
    private final KafkaTemplate<String, Object> kafkaTemplate;

    @KafkaListener(
        topics = "order-created",
        groupId = "inventory-service"
    )
    public void onOrderCreated(OrderCreatedEvent event) {

        try {
            inventoryService.reserveStock(

```

```

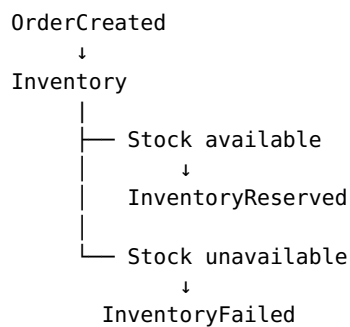
        event.getOrderId(),
        event.getItems()
    );

    kafkaTemplate.send(
        "inventory-reserved",
        new InventoryReservedEvent(
            event.getOrderId()
        )
    );

} catch (OutOfStockException e) {
    kafkaTemplate.send(
        "inventory-reservation-failed",
        new InventoryFailedEvent(
            event.getOrderId()
        )
    );
}
}
}
}

```

Flow:



5. Step 3 — Payment Service

Payment listens to:

inventory-reserved

```

@KafkaListener(
    topics = "inventory-reserved",
    groupId = "payment-service"
)
public void processPayment(
    InventoryReservedEvent event) {

    try {
        paymentService.charge(
            event.getOrderId()
        );

        kafkaTemplate.send(
            "payment-completed",
            new PaymentCompletedEvent(
                event.getOrderId()
            )
        );

    } catch (PaymentFailedException e) {
        kafkaTemplate.send(
            "payment-failed",
            new PaymentFailedEvent(
                event.getOrderId()
            )
        );
    }
}

```

```
    );  
  }  
}
```

6. Payment Failure — Compensation

Suppose:

Order
↓
Created ✓

Inventory
↓
Reserved ✓

Payment
↓
Failed ✕

Payment publishes:

payment-failed

Inventory listens:

```
@KafkaListener(  
    topics = "payment-failed",  
    groupId = "inventory-service"  
)  
public void onPaymentFailed(  
    PaymentFailedEvent event) {  
    inventoryService.releaseStock(  
        event.getOrderId()  
    );  
}
```

Now:

Payment Failed
↓
Release Inventory

Order Service can also listen:

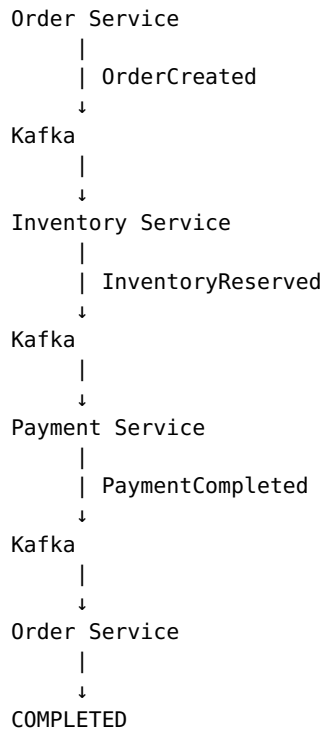
```
@KafkaListener(  
    topics = "payment-failed",  
    groupId = "order-service"  
)  
public void onPaymentFailed(  
    PaymentFailedEvent event) {  
    orderService.cancelOrder(  
        event.getOrderId()  
    );  
}
```

So:

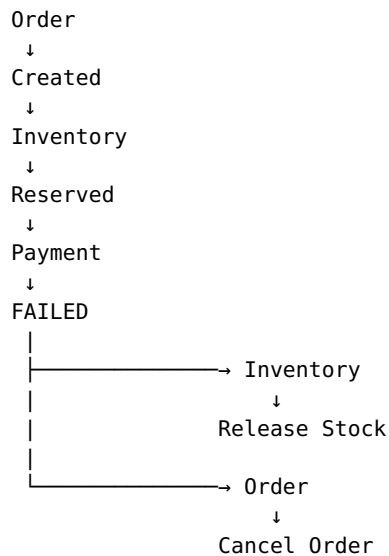
Payment Failed
|
├─→ Inventory → Release Stock
|
└─→ Order → Cancel Order

7. Complete choreography flow

Success



Failure



8. Why is it called Choreography?

Think of dancers.

There is no single person telling every dancer what to do.

Each participant knows:

"If I receive Event A,
I perform Action B
and publish Event C."

Similarly:

OrderCreated
↓

Inventory reacts

InventoryReserved

↓

Payment reacts

PaymentCompleted

↓

Order reacts

9. Main problem with Choreography

The biggest problem is **visibility**.

Imagine 10 services:

Order

↓

Inventory

↓

Payment

↓

Fraud

↓

Wallet

↓

Shipping

↓

Notification

↓

Reporting

Now someone asks:

| “Why is Order #1001 stuck?”

You may need to inspect:

Order logs

Inventory logs

Payment logs

Fraud logs

Wallet logs

Shipping logs

Kafka events

This becomes difficult.

10. How do we solve observability?

Use:

Correlation ID

+

Saga ID

+

Distributed tracing

+

Centralized logs

Example:

sagaId = SAGA-10001

orderId = ORD-10001

correlationId = CORR-ABC123

Every event contains:

```
{  
  "sagaId": "SAGA-10001",  
  "orderId": "ORD-10001",  
  "eventType": "PAYMENT_FAILED"  
}
```

Then you can trace:

SAGA-10001

```
OrderCreated  
  ↓  
InventoryReserved  
  ↓  
PaymentFailed  
  ↓  
InventoryReleased  
  ↓  
OrderCancelled
```

This is a very good **4-7 year interview point**.

11. Duplicate events

Kafka consumers can receive duplicates depending on your delivery/processing design.

Example:

```
InventoryReserved  
InventoryReserved
```

If your consumer processes both:

```
Reserve Stock  
Reserve Stock AGAIN
```

Potential problem.

Use **Idempotent Consumer**.

For example:

```
eventId = EVT-10001
```

Store processed event IDs:

```
processed_events  
-----  
event_id  
processed_at
```

Before processing:

Is EVT-10001 already processed?

```
YES → Ignore  
NO  → Process
```

12. Out-of-order events

You could receive:

```
PaymentCompleted
```

before:

InventoryReserved

depending on your architecture and event relationships.

You need to design consumers to handle invalid state transitions, persist state where necessary, and avoid assuming that event arrival order across independent topics is globally guaranteed.

13. Advantages

Advantage	Explanation
No central coordinator	No orchestrator required
Loose coupling	Services react to events
Scalable	Event-driven
Independent services	Each owns its DB
No distributed DB transaction	Uses local transactions

14. Disadvantages

Problem	Explanation
Hard to understand	Flow is distributed
Debugging difficult	Many services/events
Event cycles	Poor design can create loops
State tracking	Saga state is distributed
Compensation complexity	Every failure needs handling

15. When should you use Choreography?

Good:

Order
↓
Inventory
↓
Notification

Only a few steps and straightforward event reactions.

Less suitable:

Order
↓
Inventory
↓
Fraud
↓
Payment
↓
Wallet
↓
Shipping
↓
KYC
↓
Notification
↓
Refund

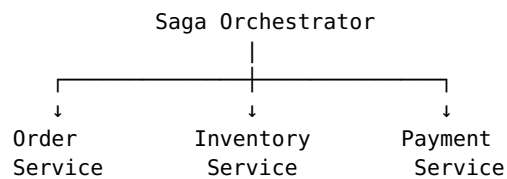
with lots of:

if
else
timeout
retry
compensation
conditional branches

At that point, **orchestration** can provide clearer control and observability.

16. SAGA — ORCHESTRATION

Orchestration introduces a central coordinator.



The orchestrator says:

1. Create Order
 2. Reserve Inventory
 3. Charge Payment
 4. Schedule Shipping
 5. Complete Order
-

17. Simple Orchestrator

```
@Service
public class OrderSagaOrchestrator {

    public void execute(Order order) {

        try {
            // Step 1
            orderService.createOrder(order);

            // Step 2
            inventoryClient.reserveStock(order);

            // Step 3
            paymentClient.charge(order);

            // Step 4
            shippingClient.schedule(order);

            // Step 5
            orderService.markCompleted(
                order.getId()
            );

        } catch (Exception e) {
            compensate(order, e);
        }
    }

    private void compensate(
        Order order,
        Exception exception) {
```

```

    }
    // Compensation logic
}

```

But this is only a **simplified interview/demo implementation**.

18. Why isn't try/catch enough in production?

Imagine:

```

Step 1 ✓
Step 2 ✓
Step 3 ✓
Step 4
↓
Orchestrator crashes

```

After restart:

Where was the saga?
 Which steps completed?
 Which steps need compensation?

A hand-written in-memory try/catch doesn't give you durable workflow state.

For complex workflows, use a durable workflow/state-management approach such as:

Temporal
 Camunda
 Axon

depending on your architecture and team choices.

19. Choreography vs Orchestration

	Choreography	Orchestration
Coordinator	No	Yes
Communication	Events	Commands/calls + events
Control	Distributed	Central
Coupling	Generally looser	Orchestrator knows participants
Debugging	Harder	Easier
Simple workflow	Good fit	Can be overkill
Complex workflow	Can become difficult	Often easier to manage
Failure handling	Distributed	Central workflow
Risk	Event cycles/hidden coupling	God-object orchestrator

20. Interview answer

What is Saga?

“Saga is a distributed transaction pattern used when a business transaction spans multiple microservices. Each service performs its own local transaction, and the overall transaction is coordinated either through choreography or orchestration. If a later step fails, compensating actions are performed for previously completed steps.”

Choreography?

“In choreography, there is no central coordinator. Each service consumes events and publishes the next event. For example, OrderCreated triggers Inventory, InventoryReserved triggers Payment, and PaymentFailed can trigger compensation events such as releasing inventory and cancelling the order.”

Orchestration?

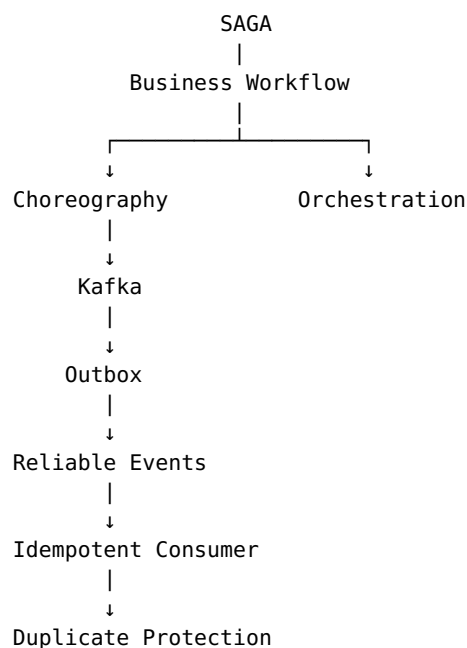
“In orchestration, a central Saga Orchestrator controls the workflow, explicitly invokes each participating service and triggers compensation when a step fails.”

When does choreography become unmanageable?

“When the workflow has many steps, complex branching, timeouts, compensation rules and cross-service dependencies. At that point, a durable orchestrator or workflow engine gives better visibility and state management.”

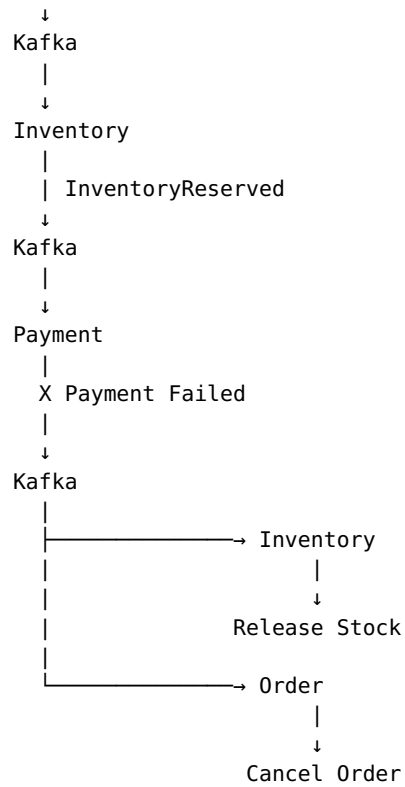
21. Very important 5+ year architecture

For your experience level, connect **Saga + Outbox + Idempotency + Kafka**:



The complete payment example

Order
|
| OrderCreated



Remember:

- Saga → Distributed business transaction
- Choreography → Services coordinate through events
- Orchestration → Central coordinator controls workflow
- Compensation → Undo/offset a previously completed business action
- Outbox → Reliable event publication
- Idempotency → Safely handle duplicate events/requests